Lekcja 4: Podstawy Pythona – Część 2

`#lekcja` `#python` `#podstawy` `#wejście-wyjście` `#pamięć`

W tej lekcji kontynuujemy naszą podróż po fundamentach Pythona. Skupimy się na interakcji z użytkownikiem, zrozumiemy, jak Python zarządza danymi "pod maską", nauczymy się konwertować typy danych i pisać kod, który jest zrozumiały nie tylko dla komputera, ale i dla innych programistów.

1. Konwersja typów (rzutowanie)

Rzutowanie (ang. *type casting*) to proces jawnej zmiany jednego typu danych na inny. Jest to jedna z najczęstszych operacji w programowaniu, ponieważ dane, które otrzymujemy (np. od użytkownika), rzadko kiedy są w formacie, którego potrzebujemy do obliczeń.

Dlaczego konwersja jest potrzebna?

- Funkcja `input()` zawsze zwraca tekst (`str`), nawet jeśli wpiszemy cyfry. Aby wykonać obliczenia matematyczne, musimy zamienić ten tekst na liczbę (`int` lub `float`).
- Chcemy połączyć tekst z liczbą w jednym komunikacie (np. `"Masz lat: " + 25` — to spowoduje błąd, trzeba zamienić `25` na `str(25)`).
- Chcemy sprawdzić, czy wartość jest "pusta" lub "niepusta" w kontekście logicznym (konwersja na `bool`).

Najważniejsze funkcje konwertujące:

Funkcja	Opis	Przykład
`int()`	Konwertuje na liczbę całkowitą.	`int("123")` -> `123`
`float()`	Konwertuje na liczbę zmiennoprzecinkową.	`float("3.14")` -> `3.14`
`str()`	Konwertuje na łańcuch znaków (tekst).	`str(100)` -> `"100"`
`bool()`	Konwertuje na wartość logiczną (`True` / `False`).	`bool("text")` -> `True`

Konwersja na `bool` – "prawdziwość" i "fałszywość"

W Pythonie każda wartość ma swoją logiczną interpretację.

- **Wartości "fałszywe" (konwertowane na `False`):**
  - Liczba `0` i `0.0`

    - Pusty tekst `""`
    - Puste kolekcje: `[]` (lista), `()` (krotka), `{}` (słownik), `set()` (zbiór)
    - Specjalna wartość `None`
- **Wszystkie inne wartości są "prawdziwe" (konwertowane na `True`).**

```
print(bool(0))      # False
print(bool(-10))    # True
print(bool(""))      # False
print(bool("Hello")) # True
print(bool([]))      # False
print(bool([1, 2]))  # True
```

Typowe błędy konwersji

Nie każdą wartość da się przekonwertować na inną. Próba zamiany tekstu, który nie wygląda jak liczba, na `int` lub `float` spowoduje błąd `ValueError`.

```
# To zadziała
liczba = int("  25  ") # Python poradzi sobie z białymi znakami

# To spowoduje błąd!
# błąd = int("dwadzieścia pięć") # ValueError: invalid literal for int()
# błąd = int("3.14")             # ValueError, bo to nie jest liczba
całkowita
```

2. Funkcje wejścia i wyjścia

`print()` – jak coś wypisać na ekran

Funkcja `print()` to nasz główny sposób komunikacji z użytkownikiem.

```
imie = "Anna"
wiek = 30
print("Cześć, świecie!")
print("Użytkownik:", imie, "Wiek:", wiek)
```

Argumenty `sep` i `end`

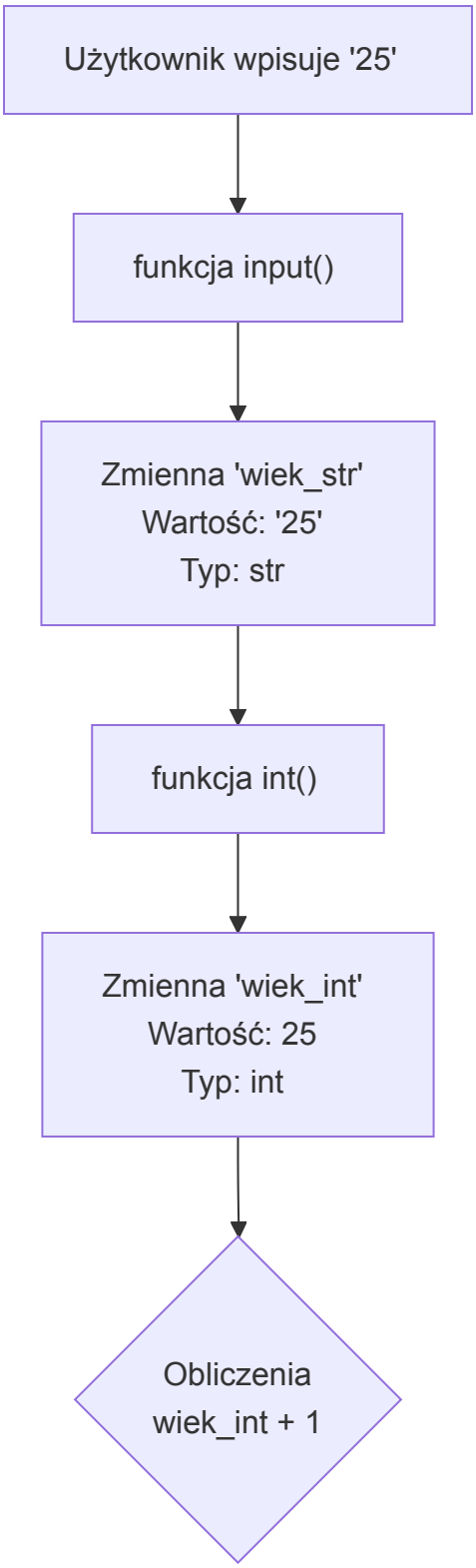
Domyślnie `print()` oddziela argumenty spacją (`sep=' '`) i kończy swoje działanie przejściem do nowej linii (`end='\n'`). Możemy to zmienić.

```python
# Zmieniamy separator na ", "
print("jabłko", "banan", "wiśnia", sep=", ")
# Wynik: jabłko, banan, wiśnia

# Zmieniamy znak końca linii na "!" i dodajemy spację
print("Pierwsza linia", end="! ")
print("Druga linia")
# Wynik: Pierwsza linia! Druga linia
```

`input()` – jak wczytać dane od użytkownika

Funkcja `input()` zatrzymuje program, wyświetla opcjonalny komunikat i czeka, aż użytkownik wpisze dane i naciśnie Enter.

> ⚠ **Warning**
>
> Złota zasada: input() zawsze zwraca dane typu str (tekst)!



Przykład praktyczny:

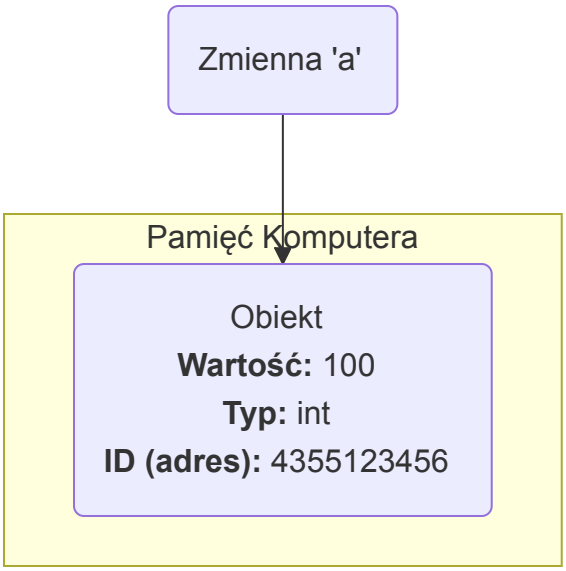
```
# 1. Pobranie danych (zawsze jako string)
wiek_str = input("Podaj swój wiek: ")

# 2. Konwersja na liczbę całkowitą
wiek_int = int(wiek_str)
```

```
# 3. Wykonanie operacji
przyszly_wiek = wiek_int + 5
print(f"Za 5 lat będziesz mieć {przyszly_wiek} lat.")
```

3. Jak zmienne są reprezentowane w pamięci?

Gdy tworzysz zmienną, np. `a = 100`, w rzeczywistości dzieją się trzy rzeczy:

1. Python tworzy w pamięci **obiekt** reprezentujący wartość `100`.
2. Ten obiekt ma przypisany **typ** (`int`) i unikalny **identyfikator** (adres w pamięci).
3. Python tworzy **etykietę** (nazwę zmiennej `a`), która wskazuje na ten obiekt.



Możemy to sprawdzić za pomocą wbudowanych funkcji:

```
a = 100
print(f"Wartość: {a}")
print(f"Typ: {type(a)}")      # <class 'int'>
print(f"ID w pamięci: {id(a)}") # np. 4355123456
```

Optymalizacja pamięci w Pythonie

Aby oszczędzać pamięć, Python często ponownie wykorzystuje istniejące obiekty dla typów niemutowalnych.

- Małe liczby całkowite (zazwyczaj od -5 do 256) są buforowane.
- Krótkie ciągi znaków również mogą być ponownie użyte.

```
x = 100
y = 100
z = 300
w = 300

# Operator `is` sprawdza, czy dwie zmienne wskazują na TEN SAM obiekt (czyli czy ich ID są równe)
print(f"ID x: {id(x)}, ID y: {id(y)}")
print(f"Czy x i y to ten sam obiekt? {x is y}") # True — Python użył tego samego obiektu dla małej liczby

print(f"ID z: {id(z)}, ID w: {id(w)}")
print(f"Czy z i w to ten sam obiekt? {z is w}") # Może być False, bo 300 to duża liczba
```

> ⚠ **Warning**
>
> Typy mutowalne (np. list, dict) zawsze tworzą nowy obiekt. Nawet jeśli wyglądają tak samo, będą miały różne adresy w pamięci.

```
lista_a = [1, 2, 3]
lista_b = [1, 2, 3]

print(f"Czy lista_a i lista_b to ten sam obiekt? {lista_a is lista_b}") # Zawsze False
print(f"Czy zawartość list jest taka sama? {lista_a == lista_b}")   # True
```

4. Komentarze w kodzie

Komentarz to fragment tekstu w kodzie, który jest całkowicie ignorowany przez interpreter Pythona. Służy do wyjaśniania, co, dlaczego i jak robi nasz program.

Kiedy pisać komentarze?

- Gdy logika kodu jest skomplikowana lub nieoczywista.
- Gdy chcesz zostawić notatkę dla siebie lub innych na przyszłość (np. `TODO: Poprawić to`).
- Gdy wyjaśniasz "dlaczego" coś zostało zrobione w dany sposób, a nie "co" jest robione.

Rodzaje komentarzy

1. Komentarz jednoliniowy (`#`)

Zaczyna się od znaku `#` i trwa do końca linii.

```
# To jest komentarz jednoliniowy
wiek = 25 # To jest komentarz w tej samej linii co kod (inline)
```

2. Docstring (łańcuch dokumentujący `"""...""")`

To specjalny rodzaj komentarza, umieszczony w potrójnych cudzysłowach. Służy do dokumentowania funkcji, modułów i klas.

```
def dodaj(a, b):
    """
    Ta funkcja przyjmuje dwie liczby i zwraca ich sumę.
    Jest to przykład wieloliniowego docstringu.
    """
    return a + b
```

Docstringi są dostępne w trakcie działania programu i mogą być użyte przez narzędzia do automatycznego generowania dokumentacji.

🧪 5. Zadania praktyczne

1. **Mini-kalkulator:** Napisz program, który prosi użytkownika o podanie dwóch liczb, a następnie wyświetla wynik ich dodawania, odejmowania, mnożenia i dzielenia. Pamiętaj o konwersji typów z `input()`.
2. **Identyfikator obiektu:** Utwórz trzy zmienne (`a`, `b`, `c`) z tą samą wartością `256`. Sprawdź i wyświetl ich `id()`. Następnie utwórz trzy zmienne z wartością `257` i również sprawdź ich `id()`. Czy widzisz różnicę w zachowaniu Pythona? Wyjaśnij dlaczego w komentarzu.
3. **Porównanie `is` vs `==`:** Utwórz dwie różne listy `lista1 = [1, 1]` i `lista2 = [1, 1]`. Sprawdź wynik porównania `lista1 is lista2` oraz `lista1 == lista2`. Wyświetl wyniki i w komentarzu wyjaśnij, dlaczego są różne.
4. **Formatowanie `print()`:** Napisz program, który wyświetli listę zakupów: `"jajka,mleko,chleb"`. Użyj funkcji `print()` z trzema argumentami tekstowymi i odpowiednio ustawionym parametrem `sep`.
5. **Wejście w jednej linii:** Użyj funkcji `print()` i jej parametru `end`, aby zadać pytanie i pozwolić użytkownikowi odpowiedzieć w tej samej linii. Przykład: `Podaj swoje imię: [tutaj użytkownik wpisuje]`.
6. **Prawda czy fałsz?:** Napisz program, który prosi użytkownika o wpisanie dowolnego tekstu. Następnie, używając konwersji na `bool`, sprawdź, czy wpisany tekst jest "prawdziwy" (niepusty) i wyświetl odpowiedni komunikat.

7. **Błąd konwersji:** Napisz program, który świadomie spróbuje przekonwertować słowo `"Python"` na liczbę całkowitą. Uruchom go, zobacz błąd `ValueError`, a następnie "napraw" program, umieszczając błędną linię w komentarzu i dodając wyjaśnienie, dlaczego kod nie działał.
8. **Obliczanie wieku psa:** Przyjmuje się, że pierwszy rok życia psa to 15 ludzkich lat, drugi to 9, a każdy kolejny to 5. Napisz program, który pyta o wiek psa w latach, a następnie oblicza i wyświetla jego wiek w "ludzkich" latach.
9. **Identyfikator po zmianie:** Utwórz zmienną `x = 10`. Wyświetl jej `id()`. Następnie przypisz do `x` nową wartość `x = x + 1`. Ponownie wyświetl `id()`. Czy identyfikator się zmienił? Dlaczego? Odpowiedz w komentarzu.
10. **Komentowanie kodu:** Poniżej znajduje się fragment kodu. Dodaj do niego komentarze jednoliniowe oraz docstring dla funkcji, wyjaśniając, co robi każda część.

```python
def oblicz_pole_prostokata(a, b):
    # Tutaj dodaj docstring

    # Tutaj dodaj komentarz
    pole = a * b

    # Tutaj dodaj komentarz
    return pole

bok_a = 10
bok_b = 20
wynik = oblicz_pole_prostokata(bok_a, bok_b)
print(f"Pole prostokąta o bokach {bok_a} i {bok_b} wynosi {wynik}.")
```